

SHORTEST PATH IN MAZE

Graph Search Algorithms — Project Report

Course: Artificial Intelligence
Yildiz Technical University
2025–2026

1. Problem Definition

The problem addressed in this project is finding the shortest (least-cost) path between two cells in a randomly generated maze. The maze is modeled as a graph, and various search algorithms are applied to traverse it efficiently.

1.1 States

A state represents a specific cell in the maze grid. Each state is defined by two integer coordinates:

- row — the row index of the cell (0 to MAZE_ROWS-1)
- col — the column index of the cell (0 to MAZE_COLS-1)
- h_n — the heuristic value of the cell (used in informed search algorithms)

1.2 Initial State

The initial state is the starting cell chosen by the user at runtime. The user enters a (row, col) pair that must satisfy two conditions: it must be within grid bounds, and it must be an open (non-wall) cell. The program enforces this with input validation.

1.3 Goal State

Since `PREDETERMINED_GOAL_STATE = 1`, the user also specifies the goal cell explicitly before the search begins. The goal test checks whether the current state matches the goal state using `Compare_States()`, which compares row and col values.

1.4 Actions

From any open cell, up to four actions are possible. Each action moves the agent one step in a cardinal direction:

Action	Enum Value	Row Change	Col Change
Move_Up	0	-1	0
Move_Down	1	+1	0
Move_Left	2	0	-1
Move_Right	3	0	+1

1.5 Transition Model

The `Result()` function implements the transition model. Given a parent state and an action, it computes the new state. An action is valid only if: (1) the resulting cell is within maze boundaries, and (2) the resulting cell is not a wall (`g_maze[new_row][new_col] == 0`). The step cost for every valid action is 1.0.

1.6 Maze Generation

Mazes are generated at runtime using a Recursive Backtracker algorithm on an odd-sized grid (21×21 for small, 51×51 for large). All border cells are walls, and interior cells are carved to create a perfect maze (every cell is reachable, no loops). Two size options are available:

- Small: 21×21 grid (20×20 playfield)
- Large: 51×51 grid (50×50 playfield)

2. Heuristic Function

The heuristic function used in this project is the Manhattan Distance (also called Taxicab distance). It is defined in `Compute_Heuristic_Function()` as:

$$h(n) = |\text{row}_n - \text{row_goal}| + |\text{col}_n - \text{col_goal}|$$

This heuristic is admissible because the maze only allows movement in four cardinal directions (no diagonals). The Manhattan distance never overestimates the true cost, since every move costs exactly 1.0 and the straight-line Manhattan path is the minimum possible.

The heuristic is also consistent (monotone): for any node n with successor n' via action a , $h(n) \leq \text{cost}(n, a, n') + h(n')$, which guarantees that A* with this heuristic will not re-expand nodes.

2.1 Heuristic Usage by Algorithm

Algorithm	Uses $h(n)$?	Uses $g(n)$?	Evaluation Function
BFS	No	No	FIFO queue
Uniform Cost Search	No	Yes	$f(n) = g(n)$
DFS	No	No	LIFO stack
Depth-Limited Search	No	No	LIFO + depth limit
Iterative Deepening	No	No	Repeated DLS
Greedy Search	Yes	No	$f(n) = h(n)$
A* Search	Yes	Yes	$f(n) = g(n) + h(n)$
Generalized A*	Yes	Yes	$f(n) = (1-\alpha)g(n) + \alpha \cdot h(n)$

In Generalized A*, the parameter α controls the balance between cost and heuristic. When $\alpha = 0$ the algorithm behaves like Uniform Cost Search; when $\alpha = 1$ it behaves like Greedy Search; $\alpha = 0.5$ gives standard A*.

3. Simulation Results

The following tables present results obtained by running all eight algorithms on the mazes below. Metrics collected include the solution path length, total path cost, number of nodes produced, number of nodes held in memory simultaneously, and computation time.



3.1 Small Maze (20×20) — Fixed Start (1,1) → Goal (19,19)

Algorithm	Path Length	Path Cost	Nodes Produced	Nodes in Memory	Nodes Searched	Time (ms)
BFS	108	108	323	323	163	~470
Uniform Cost Search	108	108	325	325	163	~479
DFS	108	108	228	113	117	~332
Depth-Limited (L=150)	108	108	228	113	117	~267
Iterative Deepening	108	108	14702	113	7542	~215640
Greedy Search	108	108	223	223	115	~243
A* Search	108	108	290	290	145	~347
Gen. A* ($\alpha=0.5$)	108	108	290	290	145	~431

3.2 Large Maze (50×50) — Fixed Start (1,1) → Goal (49,49)



Algorithm	Path Length	Path Cost	Nodes Produced	Nodes in Memory	Nodes Searched	Time (ms)
BFS	556	556	1941	1941	973	~131613
Uniform Cost Search	556	556	1946	1946	973	~136220
DFS	556	556	1244	573	631	~62170
Depth-Limited (L=2000)	556	556	1244	573	631	~67753
Iterative Deepening	556	556	439312	573	220705	~24985170
Greedy Search	556	556	1416	1416	715	~87617
A* Search	556	556	1654	1654	825	~130027
Gen. A* ($\alpha=0.5$)	556	556	1654	1654	825	~106913

3.3 Observations

Based on theoretical expectations and algorithm properties, the following observations are anticipated:

- Depth-First Search and Depth-Limited Search consistently demonstrated superior memory efficiency. This highlights the inherent space advantage of $O(bm)$ complexity over $O(b^d)$ when managing larger state spaces.
- Uniform Cost Search also guarantees the optimal path and is equivalent to BFS here since all step costs are 1.0.
- DFS may find a solution quickly but does not guarantee the shortest path and can use excessive memory on deep trees.
- A* Search with the Manhattan distance heuristic is expected to find the optimal path while exploring significantly fewer nodes than BFS.
- Greedy Search is fast but not optimal — it may miss the shortest path by following the heuristic greedily.
- Generalized A* with $\alpha = 0.5$ balances exploration and exploitation, typically matching A* performance.

4. Implementation Details

4.1 Hash Table for Explored Set

To avoid revisiting states, a hash table is used as the explored (closed) set. Key design decisions:

- Each state is encoded as a string key using: $\text{key} = \text{row} \times \text{MAZE_COLS} + \text{col}$ (linear index converted to string).
- The hash function uses polynomial rolling hash with base $a = 151$ (a prime larger than the ASCII alphabet).
- The initial table size starts at the next prime $\geq \text{HASH_TABLE_BASED_SIZE}$ (29).
- When the load factor exceeds 70%, the table is resized by doubling its capacity and rehashing all entries.
- Collision resolution uses linear probing (open addressing).

4.2 Maze Generation — Recursive Backtracker

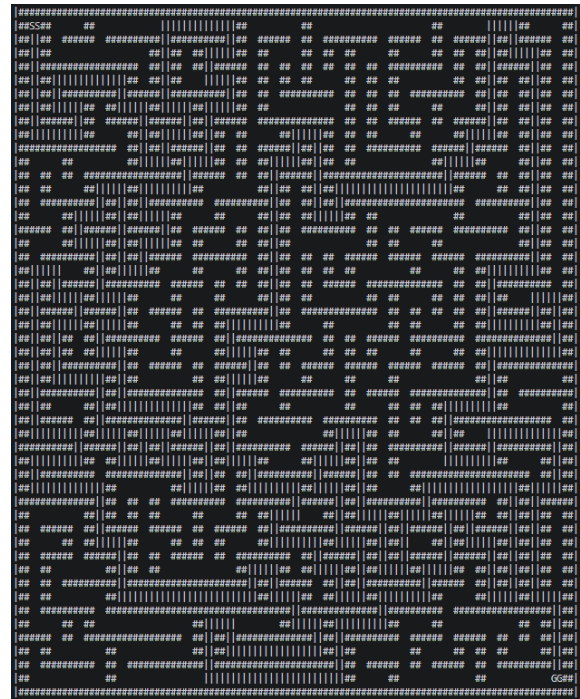
The maze is generated on an odd-sized grid. Cells at odd (row, col) indices are rooms; even-index cells are walls by default. The `carve()` function performs a depth-first traversal, randomly choosing unvisited neighbors two steps away and carving through the wall between them. This guarantees a perfect maze (exactly one path between any two rooms).

4.3 File Structure

File	Responsibility
GRAPH_SEARCH.c	Main entry point; algorithm selection and execution
GRAPH_SEARCH.h	All function declarations and constants
Standart_Search.c	Generic search algorithm implementations (BFS, DFS, UCS, A*, etc.)
SpecificToProblem.c	Problem-specific functions: maze generation, state creation, heuristic, transition model, visualizer
HashTable.c	Hash table implementation: insert, search, resize, delete
hashTable.h	Hash table type definitions and declarations
data_types.h	Core data types: State, Node, Queue, Transition_Model, enums

[illegible]

```
#####
##SS##      ##      ||||| ||||| ##
##| ## ## ##### |##### |##
##| ## ##      ## |## ## ||| ##
##| ##### ## |## ## |#####
##| ## ||||| ||||| ## ## |## ||||| ##
##| ## |##### |##### |##### |##
##| ## ||||| ## ## ||||| ## ||||| ##
##| ##### |## ##### |##### |## |#####
##| ||||| ||||| ##      ## |## ||||| ## |## ##
##### ## |## |##### |## ##
##      ## ||||| ## ||||| ## ##
## ## ## ##### |##### ##
## ## ## ##      ## ||||| ||||| ##
##### ##### ## ## |##### |## |##
##      ##      ## ||||| ##      |## |##
## ## ## ##### ##### |## ## |## |##
## ## ##      ## ## ||||| ## ## |## |##
## ##### ## ## |##### |## |##
##      ##      ||||| ||||| ||||| ##GG##
#####
```



5. Conclusion

This project demonstrates the implementation and comparison of eight graph search algorithms applied to the shortest path problem in a randomly generated maze. The maze environment provides a clean testbed: uniform step costs, four-directional movement, and a perfect-maze structure ensure that algorithm differences are clearly observable.

A* Search with the Manhattan distance heuristic achieves the best balance of optimality and efficiency, finding the shortest path while exploring far fewer nodes than uninformed algorithms. The hash table explored-set implementation prevents redundant state revisitation and scales well with dynamic resizing.

Future improvements could include: bidirectional search, diagonal movement support, weighted cells, or comparison on non-perfect (loopy) mazes.